15

Harness em Projetos Legacy

A maioria das software houses brasileiras tem pelo menos um: o sistema com décadas de história, mais de mil units numa pasta só, sem estrutura visível, sem testes automatizados, com acoplamento alto entre tudo. Quem tenta usar IA nesse cenário sem preparação para de usar em uma semana — e conclui que "IA não funciona para legacy". Este material mostra a sequência que funciona.

A IDEIA CENTRAL DESTE MATERIAL

O objetivo não é usar IA para reescrever o sistema. É usar IA para finalmente entender o sistema — e fazer mudanças cirúrgicas com confiança. Em projeto novo, o Harness começa pelo SPEC; em legacy, começa pelo discovery: mapear o que existe.

1 Por que a IA "não funciona" em legacy

Sem preparação, o roteiro do fracasso é sempre o mesmo:

- O agent tenta ler tudo → **estoura a janela de contexto** → se perde sem estrutura lógica
- Você pede uma correção → o modelo não conhece as dependências → corrige o bug → **quebra três units** que você não viu
- Pede outra correção → o modelo não lembra do que foi feito → **desfaz a correção anterior** sem perceber

Resultado: você desiste. Mas o problema não é o modelo — é a ausência de Harness. É o whack-a-mole do material 03 elevado à décima potência, porque aqui as dependências invisíveis se contam às centenas.

2 Projeto novo vs projeto legacy

	PROJETO NOVO	PROJETO LEGACY
Ponto de partida	Tela em branco	Sistema existente, com história
Convenções	Você define antes	Implícitas, não documentadas
Decisões	Tomadas agora	Tomadas por gente que já saiu da empresa
Acoplamento	Controlado desde o início	Invisível entre componentes
Harness começa em...	SPEC.md (material 05)	**Discovery** — mapear o que existe

3 A sequência completa, passo a passo

O exemplo usa Delphi — o legacy clássico das software houses — mas a sequência vale para qualquer stack antiga.

Passo 1 — .claudeignore agressivo

Com 1000+ arquivos numa pasta, a primeira providência é dizer o que o agent **nunca** deve ler — compilados, backups, temporários. Isso elimina metade do ruído antes da primeira sessão (é o material 06 aplicado com força máxima):

```
# .claudeignore — legacy Delphi
# Compilados e gerados
*.dcu  *.dpu  *.exe  *.dll  *.bpl  *.res  *.map
__history/
__recovery/
*.~*
# Backups
*.bak  *.old  *.orig
# Logs e temporários
*.log  *.tmp
```

Passo 2 — Discovery: o agent mapeia o sistema

Não tente escrever o SPEC.md do zero — mande o agent descobrir:

▸ PROMPT DE DISCOVERY

```
Leia todos os arquivos .pas e .dfm desta pasta. Não edite nada.

Gere um mapa do sistema com:
1. Módulos lógicos identificados
   (agrupe units por responsabilidade aparente)
2. As 20 units mais referenciadas por outras
   (os núcleos críticos do sistema)
3. Units que parecem standalone
   (candidatas a modificação mais segura)
4. Dependências entre os grupos identificados
5. O que o sistema parece fazer
   (resumo de negócio baseado no código)

Salve em SPEC.md
```

Esse mapa guia tudo que vem depois. Sem ele, você está operando às cegas — de novo.

Passo 3 — CLAUDE.md dominado por restrições

Em projeto novo, o CLAUDE.md descreve convenções. Em legacy, ele descreve **o que nunca pode acontecer**:

```
RESTRIÇÕES CRÍTICAS:
- Nunca mover arquivos sem aprovação explícita
- Nunca renomear units sem aprovação explícita
- Nunca modificar mais de uma unit por sessão
- Nunca editar units críticas sem teste manual confirmado
- Nunca alterar interfaces públicas sem mapear todos os usos

Estratégia de teste: manual — descrever O QUE testar
antes de qualquer edição

Regra de ouro: uma unit por sessão, escopo mínimo,
efeito colateral zero
```

Passo 4 — Mapa de dependências com níveis de risco

As 20 units mais referenciadas são o núcleo: mudanças nelas têm efeitos em cascata invisíveis. Peça ao agent, para cada uma: o que faz, de quem depende, **quem depende dela**, interfaces públicas — e um nível de risco. Salve em `.claude/knowledge/dependency-map.md`:

NÍVEL	CRITÉRIO	PROTOCOLO
ALTO	Usada por > 20 units	Nunca editar sem mapeamento completo de todos os usos
MÉDIO	Usada por 5–20 units	Listar todos os usos antes de qualquer edição
BAIXO	Usada por < 5 units	Modificação mais segura — ainda assim com teste manual

Esse arquivo é consultado antes de qualquer modificação. É o que impede a "decisão razoável" que quebra dez units.

Passo 5 — Rules e Hooks de proteção

MECANISMO	CONFIGURAÇÃO
Rule: units críticas	Escopo: as 20 units do mapa. Instrução: antes de editar, listar todos os usos da função modificada.
Rule: no-move	Escopo: toda a pasta /src. Instrução: nunca mover, renomear ou deletar sem ordem explícita.
Hook: PreToolUse em unit crítica	Exibe quantas units dependem desta e pede confirmação antes de prosseguir.
Hook: Move/Rename/ Delete	Bloqueio total com exit 2 (material 11).
Hook: Stop	Gera o checklist manual do que testar, baseado nas units editadas na sessão.

Passo 6 — RAG do histórico

A cada sessão de manutenção, capture: qual unit foi modificada, qual era o problema, o que foi tentado e falhou, a solução final, quais units o teste manual cobriu, alertas para o futuro. Salve em `.claude/knowledge/` e faça o ingest no RAG (material 14). Com o tempo, esse histórico vira **a memória que o sistema nunca teve** — tudo que já quebrou, como foi resolvido, o que nunca tentar de novo.

Passo 7 — PLAN.md por módulo lógico

Planejamento com escopo mínimo: uma unit por task, teste manual definido **antes** de qualquer edição (material 07 adaptado ao risco).

4 A sequência em uma tabela

#	ETAPA	O QUE GARANTE
1	.claudeignore agressivo	Janela de contexto protegida do ruído
2	Discovery → SPEC.md	Mapa do sistema gerado pelo próprio agent
3	CLAUDE.md de restrições	O que nunca pode acontecer, por escrito
4	Mapa de dependências	As units de alto risco identificadas
5	Rules + Hooks	Proteção determinística contra mudança destrutiva
6	RAG do histórico	Memória que cresce a cada sessão
7	PLAN.md por módulo	Escopo mínimo, teste manual antes de editar

5 A regra de ouro do legacy

▸ UMA UNIT POR SESSÃO. ESCOPO MÍNIMO. TESTE MANUAL ANTES DE ENCERRAR.

Com 1000+ units acopladas, a tentação é resolver vários problemas de uma vez — e essa tentação é o maior risco do projeto inteiro. O Harness cria **fricção intencional** contra ela: não para travar o desenvolvimento, mas para garantir que cada mudança é entendida, testada e documentada antes da próxima começar.

✗ ANTES DO HARNESS

Modelo cego + legacy complexo.

Whack-a-mole infinito — até você desistir da IA.

✓ DEPOIS DO HARNESS

Modelo com o contexto do sistema.

Uma mudança controlada por vez, com memória do que já foi tentado e proteção do que não pode mudar.

▸ **COLA PARA FIXAR**

- Legacy não começa pelo SPEC — começa pelo **discovery**: o agent mapeia o que existe.
- Ordem: **ignore agressivo → discovery → restrições → mapa de risco → rules/hooks → RAG → plano**.
- CLAUDE.md de legacy é dominado por **proibições**, não por convenções.
- As **20 units mais referenciadas** são o núcleo: risco ALTO = nunca editar sem mapear todos os usos.
- Regra de ouro: **uma unit por sessão, escopo mínimo, teste manual antes de encerrar**.
- A meta não é reescrever o sistema — é **entendê-lo** e mudar com cirurgia, não com trator.